

Activity 2 - Validate a product code with strings

AI Vibe Coding with Rust / TGS-2023039924 / version 3.0 / K1/A1

Specification

`is_valid_sku(&str) -> bool`: after trimming, exactly 3 ASCII upper-case letters, "-", 4 ASCII digits.
`normalise_sku(&str) -> Option<String>`: trimmed upper-case SKU when valid, else None.

Plan before code

Act as a senior Rust reviewer. Specification: `is_valid_sku(&str) -> bool`: after trimming, exactly 3 ASCII upper-case letters, "-", 4 ASCII digits. `normalise_sku(&str) -> Option<String>`: trimmed upper-case SKU when valid, else None. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Validate a product code with strings" in `starter/src` so that the supplied tests pass. Borrow `&str` inputs, return an owned `String` only from `normalise_sku`, and do not use regex crates. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

Review checklist

Signatures unchanged; no unsafe; no `unwrap()` on input; no new crates; every boundary tested; cargo test, `clippy -D warnings` and `fmt --check` pass; you can explain every line.